

University
Politehnica
of Bucharest





PL/SQL (V)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro



Overview

- Subprograms



Subprograms

- A PL/SQL subprogram is a named PL/SQL block that can be invoked repeatedly
- If the subprogram has parameters, their values can differ for each invocation
- A subprogram is either a
  - **Procedure** used to perform an action
  - **Function** used to compute and return a value
- Subprograms support the development and maintenance of reliable and reusable code



Subprograms Features

- Modularity
  - Subprograms let you break a program into manageable, well-defined modules
- Easier Application Design
  - Implementation details of the subprograms can be deferred until the main program is tested
  - The programs can be refined one step at a time
  - Easier to test modules independently
- Maintainability
  - The implementation details of a subprogram can be changed without changing its invokers
- Packageability
  - Subprograms can be grouped into packages



Subprograms Features

- Reusability
  - Any number of applications, in many different environments, can use the same package subprogram or standalone subprogram
- Better Performance
  - Each subprogram is compiled and stored in executable form, which can be invoked repeatedly
  - Because stored subprograms run in the database server, a single invocation over the network can start a large job
  - This division of work
    - Reduces network traffic
    - Improves response times
  - Stored subprograms are cached and shared among users
    - Lowers memory requirements
    - Lowers invocation overhead



Subprograms Types

- Subprogram can be
  - Nested subprograms:
    - Created inside the WITH clause
    - Created inside a PL/SQL block (which can be another subprogram)
  - Package subprograms: created inside a package
  - Standalone subprograms: are created at schema level



Nested Subprograms

- A subprogram created inside a PL/SQL block or WITH clause is a nested subprogram
- In the WITH clause
  - Declare **functions** that will be used in the SQL statement
  - Declare **procedures** that can only be used in subprograms
- Nested subprograms can be
  - Declared and defined at the same time
  - First declared and then defined in the same block
- A nested subprogram is stored in the database only if it is nested in
  - A standalone subprogram
  - Package subprogram



Nested Subprograms

```
 1 WITH
 2   FUNCTION calc_venit(eid employees.employee_id%type)
 3   RETURN FLOAT
 4   IS
 5     sal employees.salary%type;
 6     com employees.commission_pct%type;
 7   BEGIN
 8     SELECT salary, commission_pct
 9     INTO sal, com
10     FROM employees
11     WHERE employee_id = eid;
12     RETURN sal * (1 + nvl(com, 0));
13   END;
14 SELECT
15   first_name||' '||last_name nume,
16   calc_venit(employee_id) venit
17 FROM employees;
18 /
```



Nested Subprograms

```
WITH
  PROCEDURE procent_comm(eid employees.employee_id%type, procent NUMBER)
  IS
    name varchar2(50);
  BEGIN
    SELECT first_name||' '||last_name INTO name
    FROM employees WHERE employee_id=eid;
    DBMS_OUTPUT.put_line(name || ' procent=' || procent);
  END;

  FUNCTION calc_venit(eid employees.employee_id%type)
  RETURN FLOAT
  IS
    sal employees.salary%type;
    com employees.commission_pct%type;
  BEGIN
    SELECT salary, commission_pct INTO sal, com
    FROM employees WHERE employee_id = eid;
    procent_comm(eid, 1 + nvl(com, 0));
    RETURN sal * (1 + nvl(com, 0))
  END;
SELECT
  first_name||' '||last_name nume,
  calc_venit(employee_id) venit
FROM employees;
/
```



Package Subprograms

- A subprogram created inside a package is a **package subprogram**

- Developers can
  - Declare it in the package specification
  - Define it in the package body

- It is stored in the database until the package is dropped



Standalone Subprograms

- **A subprogram created at schema level is a standalone subprogram**
- A developer can create it with the statements:

CREATE [OR REPLACE] FUNCTION function_name
CREATE [OR REPLACE] PROCEDURE procedure_name

- It is stored in the database until it is dropped with the statements:

DROP FUNCTION function_name;
DROP PROCEDURE procedure_name;



Stored Subprograms

- A stored subprogram is either
  - A package subprogram
  - A standalone subprogram
- A stored subprogram is affected by the AUTHID and ACCESSIBLE BY clauses
- These clauses can appear in the CREATE FUNCTION, CREATE PROCEDURE, and CREATE PACKAGE statements
- The AUTHID clause affects the name resolution and privilege checking of SQL statements that the subprogram issues at run time
- The ACCESSIBLE BY clause specifies a white list of PL/SQL units that can access the subprogram



Subprogram Invocations

- A subprogram invocation has this form:

subprogram_name [([parameter [, parameter]...])]

- If the subprogram has no parameters, or specifies a default value for every parameter
  - Omit the parameter list
  - Specify an empty parameter list

- A procedure invocation is a PL/SQL statement:

procedure_name(val_parameter_1, val_parameter_2, ...);

- A function invocation is an expression:

value := function_name(val_parameter_1, val_parameter_2, ...);

IF function_name(val_parameter_1, val_parameter_2, ...) THEN ...



Subprogram Properties

- Each subprogram property (e.g., AUTHID, ACCESSIBLE BY, etc.) can appear only once in the subprogram declaration
- The properties can appear in any order
- Properties appear before the IS or AS keyword in the subprogram heading
- The properties cannot appear in nested subprograms
- Only the ACCESSIBLE BY property can appear in package subprograms



Subprogram Parts

- A subprogram begins with a subprogram heading, which specifies
  - The subprogram name
  - The subprogram parameter list (optional)

- Like an anonymous block, a subprogram has these parts:
  - Declarative part (optional)
    - This part declares and defines local types, cursors, constants, variables, exceptions, and nested subprograms
    - These items cease to exist when the subprogram completes execution
    - This part can also specify pragmas
    - The declarative part of a subprogram does not begin with the keyword DECLARE, as the declarative part of an anonymous block does
  - Executable part (required)
    - This part contains one or more statements that assign values, control execution, and manipulate data
  - Exception-handling part (optional)
    - This part contains code that handles runtime errors



Additional Subprogram Parts For Functions

- A function has the same structure as a procedure, except that:

1. A function heading must include a RETURN clause
   - The RETURN clause specifies the data type of the value that the function returns
   - A procedure heading cannot have a RETURN clause

2. In the executable part of a function, every execution path must lead to a RETURN statement
   - The PL/SQL compiler issues a compile-time warning otherwise
   - The RETURN statement is optional and not recommended in a procedure



Additional Subprogram Parts For Functions

- A function declaration can include these options:

Option	Description
DETERMINISTIC option	Helps the optimizer avoid redundant function invocations.
PARALLEL_ENABLE option	Enables the function for parallel execution, making it safe for use in slave sessions of parallel DML evaluations.
PIPELINED option	Makes a table function pipelined, for use as a row source.
RESULT_CACHE option	Stores function results in the PL/SQL function result cache.



Additional Subprogram Parts For Functions

```
 1 CREATE OR REPLACE
 2 FUNCTION f_pow(x IN NUMBER, y IN NUMBER)
 3     RETURN NUMBER
 4     DETERMINISTIC
 5     PARALLEL_ENABLE
 6     RESULT_CACHE
 7 IS
 8 BEGIN
 9   RETURN x**y;
10 END;
11 /
12
13 SELECT f_pow(2, 10) from dual;
14
15 DROP FUNCTION f_pow;
```



Additional Subprogram Parts For Functions

```
 1 CREATE TYPE info_row AS OBJECT (
 2   id   NUMBER,
 3   info VARCHAR2(50)
 4 );
 5 /
 6
 7 CREATE TYPE info_tab IS TABLE OF info_row;
 8 /
 9
10 CREATE OR REPLACE FUNCTION get_info(n IN NUMBER)
11   RETURN info_tab PIPELINED
12 AS
13 BEGIN
14   FOR i IN 1 .. n LOOP
15     PIPE ROW(info_row(i, 'Description for ' || i));
16   END LOOP;
17 END;
18 /
19
20 SELECT * FROM  TABLE(get_info(10));
21
22 DROP FUNCTION get_info;
23 DROP TYPE info_tab;
24 DROP TYPE info_row;
```



Subprogram Parts
Return

- RETURN Statement
  - The RETURN statement immediately ends the execution of the subprogram or anonymous block that contains it
  - A subprogram or anonymous block can contain multiple RETURN statements

- RETURN Statement in Function
  - Every execution path must lead to a RETURN statement and every RETURN statement must specify an expression
  - The RETURN statement assigns the value of the expression to the function identifier and returns control to the invoker, where execution resumes immediately after the invocation
  - In a pipelined table function, a RETURN statement need not specify an expression



Subprogram Parts
Return

```
 1 DECLARE
 2   x INTEGER;
 3
 4   FUNCTION f (n INTEGER) RETURN INTEGER
 5   IS
 6   BEGIN
 7     RETURN (n*n);
 8   END;
 9
10 BEGIN
11   DBMS_OUTPUT.PUT_LINE ('f returns ' || f(2) || '. Execution returns here (1).' );
12   x := f(3);
13   DBMS_OUTPUT.PUT_LINE('Execution returns here (2).');
14   x := &x;
15   IF f(x) < 10 THEN
16     DBMS_OUTPUT.PUT_LINE('Execution returns here (3).');
17   ELSE
18     DBMS_OUTPUT.PUT_LINE('Execution returns here (4).');
19   END IF;
20 END;
21 /
```



Subprogram Parts
Return

- RETURN Statement in Procedure
  - The RETURN statement returns control to the invoker, where execution resumes immediately after the invocation
  - The RETURN statement cannot specify an expression

- RETURN Statement in Anonymous Block
  - The RETURN statement exits its own block and all enclosing blocks
  - The RETURN statement cannot specify an expression



Subprogram Parts
Return

```
 1 DECLARE
 2   PROCEDURE p IS
 3   BEGIN
 4     DBMS_OUTPUT.PUT_LINE('Inside p');
 5     RETURN;
 6     DBMS_OUTPUT.PUT_LINE('Unreachable statement.');
 7   END;
 8 BEGIN
 9   p;
10   DBMS_OUTPUT.PUT_LINE('Control returns here.');
11 END;
12 /
```



Subprogram Parts
Return

```
 1 BEGIN
 2   BEGIN
 3     DBMS_OUTPUT.PUT_LINE('Inside inner block.');
 4     RETURN;
 5     DBMS_OUTPUT.PUT_LINE('Unreachable statement.');
 6   END;
 7   DBMS_OUTPUT.PUT_LINE('Inside outer block.
 8                         Unreachable statement.');
 9 END;
10 /
```



Forward Declarations

- If nested subprograms in the same PL/SQL block invoke each other, then one requires a forward declaration, because a subprogram must be declared before it can be invoked.
- A forward declaration declares a nested subprogram but does not define it
- A developer must define it later in the same block
- The forward declaration and the definition must have the same subprogram heading



Forward Declarations

```
 1 DECLARE
 2   -- Declare proc1 (forward declaration)
 3   PROCEDURE proc1;
 4
 5   -- Declarea and define proc2
 6   PROCEDURE proc2
 7   IS
 8   BEGIN
 9     DBMS_OUTPUT.PUT_LINE('Call proc1');
10     proc1;
11   END;
12
13   -- Define proc1
14   PROCEDURE proc1
15   IS
16   BEGIN
17     DBMS_OUTPUT.PUT_LINE('Inside proc1');
18   END;
19 BEGIN
20   proc2;
21 END;
22 /
```



Subprogram Parameters

- If a subprogram has parameters, their values can differ for each invocation
- Formal parameters must be declared in the subprogram heading to have subprogram with parameters
- In each formal parameter declaration specify
  - Mandatory: the name and data type of the parameter
  - Optional: the mode and default value
- Reference the formal parameters by their names in the execution part of the subprogram



Subprogram Parameters

- When invoking the subprogram, specify the actual parameters whose values are to be assigned to the formal parameters
- **Corresponding actual and formal parameters** must have compatible **data types**
- To avoid confusion, use different names for formal and actual parameters
- Actual parameters (including default values of formal parameters) can be evaluated in any order
- LOB parameters cannot be used in a server-to-server remote procedure call (RPC)



Subprogram Parameters

```
 1 DECLARE
 2   emp_num NUMBER(6) := 120;
 3   bonus   NUMBER(6) := 100;
 4   merit   NUMBER(4) := 50;
 5
 6   PROCEDURE raise_salary (
 7     emp_id NUMBER,  -- formal parameter
 8     amount NUMBER   -- formal parameter
 9   ) IS
10   BEGIN
11     UPDATE employees
12     SET salary = salary + amount  -- reference to formal parameter
13     WHERE employee_id = emp_id;   -- reference to formal parameter
14   END raise_salary;
15
16 BEGIN
17   raise_salary(emp_num, bonus);           -- actual parameters
18   raise_salary(emp_num, merit + bonus);   -- actual parameters
19 END;
20 /
```



Subprogram Parameters

- If the data type of a formal parameter is a constrained subtype, then:
  - If the subtype has the NOT NULL constraint, then the actual parameter inherits it
  - If the subtype has the base type VARCHAR2, then the actual parameter does not inherit the size of the subtype
  - If the subtype has a numeric base type, then the actual parameter inherits the range of the subtype, but not the precision or scale

- In a function
  - The clause RETURN datatype declares a hidden formal parameter
  - The statement RETURN value specifies the corresponding actual parameter
  - The same rules apply for subtype data types in RETURN



Subprogram Parameters

```
 1 DECLARE
 2   -- Declare subtype
 3   SUBTYPE license IS VARCHAR2(7) NOT NULL;
 4   n license := 'DLLLDDD';
 5
 6   PROCEDURE p (x License) IS
 7   BEGIN
 8     DBMS_OUTPUT.PUT_LINE(x);
 9   END;
10
11 BEGIN
12   p('1ABC123456789');   -- Succeeds; size is not inherited
13   p(NULL);              -- Raises error; NOT NULL is inherited
14 END;
15 /
```



Subprogram Parameters

```
 1 DECLARE
 2   FUNCTION test (p INTEGER) RETURN INTEGER IS
 3   BEGIN
 4     DBMS_OUTPUT.PUT_LINE('p = ' || p);
 5     RETURN p;
 6   END test;
 7
 8 BEGIN
 9   -- actual parameter and return value inherit
10   -- only range from subtype
11   DBMS_OUTPUT.PUT_LINE('test(p) = ' || test(0.66));
12 END;
13 /
```



Subprogram Parameters

```
 1 DECLARE
 2   FUNCTION test (p NUMBER) RETURN NUMBER IS
 3     -- Implicitly converts p to INTEGER
 4     q INTEGER := p;
 5   BEGIN
 6     -- Display q, not p
 7     DBMS_OUTPUT.PUT_LINE('p = ' || q);
 8     -- Return q, not p
 9     RETURN q;
10   END test;
11 BEGIN
12   DBMS_OUTPUT.PUT_LINE('test(p) = ' || test(0.66));
13 END;
14 /
```



Subprogram Parameters Passing Method

- The PL/SQL compiler has two ways of passing an actual parameter to a subprogram:
  - By reference
    - The compiler passes the subprogram a pointer to the actual parameter
    - The actual and formal parameters refer to the same memory location
  - By value
    - The compiler assigns the value of the actual parameter to the corresponding formal parameter
    - The actual and formal parameters refer to different memory locations
- The method by which the compiler passes a specific actual parameter depends on its mode



Subprogram Parameters Passing Method

- If necessary, the compiler implicitly converts the data type of the actual parameter to the data type of the formal parameter

- Avoid implicit data conversion, in either of these ways:
  - Declare the variables that you intend to use as actual parameters with the same data types as their corresponding formal parameters
  - Explicitly convert actual parameters to the data types of their corresponding formal parameters, using the SQL conversion functions described in Oracle Database SQL Language Reference



Subprogram Parameters Passing Method

```
 1 CREATE OR REPLACE PROCEDURE p (n NUMBER)
 2 AUTHID DEFINER
 3 IS
 4 BEGIN
 5   DBMS_OUTPUT.PUT_LINE(n);
 6 END;
 7 /
 8
 9 DECLARE
10   x NUMBER       :=  1;
11   y VARCHAR2(1) := '1';
12 BEGIN
13   -- No conversion needed
14   p(x);
15   -- y implicitly converted from VARCHAR2 to NUMBER
16   p(y);
17   -- y explicitly converted from VARCHAR2 to NUMBER
18   p(TO_NUMBER(y));
19 END;
20 /
```



Subprogram Parameters Mode

- The **mode** of a formal parameter determines its behavior

Parameter Mode	Is Default?	Role
IN	Default mode	Passes a value to the subprogram.
OUT	Must be specified.	Returns a value to the invoker.
IN OUT	Must be specified.	Passes an initial value to the subprogram and returns an updated value to the invoker.



Subprogram Parameters Mode Characteristics

Mode	Formal Parameter	Actual Parameter	Passed by Reference?
IN	**Formal parameter acts like a constant**: When the subprogram begins, its value is that of either its actual parameter or default value, and the subprogram cannot change this value.	Actual parameter can be a constant, initialized variable, literal, or expression.	Actual parameter is passed by reference.
OUT	**Formal parameter is initialized to the default value of its type.** The default value of the type is NULL except for a record type with a non-NULL default value. When the subprogram begins, the formal parameter has its initial value regardless of the value of its actual parameter. Oracle recommends that the subprogram assign a value to the formal parameter.	If the default value of the formal parameter type is NULL, then the actual parameter must be a variable whose data type is not defined as NOT NULL.	By default, actual parameter is passed by value. If you specify NOCOPY, it might be passed by reference.
IN OUT	**Formal parameter acts like an initialized variable**: When the subprogram begins, its value is that of its actual parameter. Oracle recommends that the subprogram update its value.	Actual parameter must be a variable (typically, it is a string buffer or numeric accumulator).	By default, actual parameter is passed by value (in both directions). If you specify NOCOPY, it might be passed by reference.



Subprogram Parameters Mode

- Do not use OUT and IN OUT for function parameters
  - Ideally, a function takes zero or more parameters and returns a single value
  - A function with IN OUT parameters returns multiple values and has side effects

- Regardless of how an OUT or IN OUT parameter is passed:
  - If the subprogram exits successfully, then the value of the actual parameter is the final value assigned to the formal parameter
  - The formal parameter is assigned at least one value (the initial value)
  - If the subprogram ends with an exception, then the value of the actual parameter is undefined
  - Formal OUT and IN OUT parameters can be returned in any order



Subprogram Parameters Mode

- When an OUT or IN OUT parameter is passed by reference:
  - The actual and formal parameters refer to the same memory location
  - If the subprogram changes the value of the formal parameter, the change shows immediately in the actual parameter



Subprogram Parameters Mode

```
 1 CREATE OR REPLACE PROCEDURE p (
 2   a         PLS_INTEGER,  -- IN by default
 3   b      IN PLS_INTEGER,
 4   c     OUT PLS_INTEGER,
 5   d IN OUT PLS_INTEGER
 6 )
 7 IS
 8 BEGIN
 9   DBMS_OUTPUT.PUT_LINE('Inside procedure p:');
10   -- Print values of parameters:
11   DBMS_OUTPUT.PUT_LINE('IN a = '     || NVL(TO_CHAR(a), 'NULL'));
12   DBMS_OUTPUT.PUT_LINE('IN b = '     || NVL(TO_CHAR(b), 'NULL'));
13   DBMS_OUTPUT.PUT_LINE('OUT c = '    || NVL(TO_CHAR(c), 'NULL'));
14   DBMS_OUTPUT.PUT_LINE('IN OUT d = ' || TO_CHAR(d));
15   -- Can reference IN parameters a and b,
16   -- but cannot assign values to them.
17   c := a + 10;  -- Assign value to OUT parameter
18   d := b * 10;  -- Assign value to IN OUT parameter
19 END;
20 /
```



Subprogram Parameters Mode

```
DECLARE
  aa  CONSTANT PLS_INTEGER := 1;
  bb  PLS_INTEGER := 2;
  cc  PLS_INTEGER := 3;
  dd  PLS_INTEGER := 4;
  ee  PLS_INTEGER;
  ff  PLS_INTEGER := 5;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Before invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('aa = ' || NVL(TO_CHAR(aa), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('bb = ' || NVL(TO_CHAR(bb), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('cc = ' || NVL(TO_CHAR(cc), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('dd = ' || TO_CHAR(dd));

  p (aa, -- constant
     bb, -- initialized variable
     cc, -- initialized variable
     dd  -- initialized variable
  );

  DBMS_OUTPUT.PUT_LINE('After invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('aa = ' || NVL(TO_CHAR(aa), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('bb = ' || NVL(TO_CHAR(bb), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('cc = ' || NVL(TO_CHAR(cc), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('dd = ' || TO_CHAR(dd));

  DBMS_OUTPUT.PUT_LINE('Before invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('ee = ' || NVL(TO_CHAR(ee), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('ff = ' || TO_CHAR(ff));

  p (1,         -- literal
     (bb+3)*4, -- expression
     ee,        -- uninitialized variable
     ff         -- initialized variable
   );

  DBMS_OUTPUT.PUT_LINE('After invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('ee = ' || NVL(TO_CHAR(ee), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('ff = ' || TO_CHAR(ff));
END;
/
```



Subprogram Parameters Aliasing

- Aliasing is having two different names for the same memory location
- If a stored item is visible by more than one path and if the item is changed by one path, then the change propagates to all paths
- Subprogram parameter aliasing always occurs when
  - The compiler passes an actual parameter by reference
  - A subprogram has cursor variable parameters
- When the compiler passes an actual parameter by reference:
  - The actual and formal parameters refer to the same memory location
  - The change shows immediately in the actual parameter if the subprogram changes the value of the formal parameter



Subprogram Parameters Aliasing

- The compiler always passes IN parameters by reference:
  - The resulting aliasing cannot cause problems
  - Subprograms cannot assign values to IN parameters

- If you specify NOCOPY for an OUT or IN OUT parameter
  - The compiler might pass an OUT or IN OUT parameter by reference
  - NOCOPY is only a hint: each time the subprogram is invoked, the compiler decides, silently, whether to obey or ignore NOCOPY.
  - Aliasing can occur for one invocation but not another, making subprogram results indeterminate



Subprogram Parameters Aliasing

- If the actual parameter is a global variable, then an assignment to the formal parameter might show in the global parameter
- If the same variable is the actual parameter for two formal parameters, then an assignment to either formal parameter might show immediately in both formal parameters
- If the actual parameter is a package variable, then an assignment to either the formal parameter or the package variable might show immediately in both the formal parameter and the package variable
- If the subprogram is exited with an unhandled exception, then an assignment to the formal parameter might show in the actual parameter



Subprogram Parameters Aliasing

```
 1 DECLARE
 2   n NUMBER := 10;
 3
 4   PROCEDURE p (
 5     n1 IN NUMBER,
 6     n2 IN OUT NUMBER,
 7     n3 IN OUT NOCOPY NUMBER
 8   ) IS
 9   BEGIN
10     -- actual parameter is 20 only after procedure succeeds
11     n2 := 20;
12     -- actual parameter value is still 10
13     DBMS_OUTPUT.put_line(n1);
14     -- might change actual parameter immediately
15     n3 := 30;
16     -- actual parameter value is either
17     -- 10 (ignore NOCOPY)
18     -- 30 (obey NOCOPY)
19     DBMS_OUTPUT.put_line(n1);
20   END;
21
22 BEGIN
23   p(n, n, n);
24   DBMS_OUTPUT.put_line(n);
25 END;
26 /
```



Subprogram Parameters
Aliasing with Cursor Variable Parameters

- Cursor variable parameters are pointers
- If a subprogram assigns one cursor variable parameter to another, they refer to the same memory location
- This aliasing can have unintended results



Subprogram Parameters Aliasing with Cursor Variable Parameters

```
DECLARE
  c1 SYS_REFCURSOR;
  c2 SYS_REFCURSOR;

  PROCEDURE get_emp_data (
    emp_cv1 IN OUT SYS_REFCURSOR,
    emp_cv2 IN OUT SYS_REFCURSOR
  )
  IS
    emp_rec employees%ROWTYPE;
  BEGIN
    OPEN emp_cv1 FOR SELECT * FROM employees;
    -- now both variables refer to same location
    emp_cv2 := emp_cv1;
    -- fetches first row of employees
    FETCH emp_cv1 INTO emp_rec;
    -- fetches second row of employees
    FETCH emp_cv1 INTO emp_rec;
    -- fetches third row of employees
    FETCH emp_cv2 INTO emp_rec;
    -- closes both variables
    CLOSE emp_cv1;
    -- causes error when get_emp_data is invoked
    -- FETCH emp_cv2 INTO emp_rec;
  END;
BEGIN
  get_emp_data(c1, c2);
END;
/
```



Subprogram Parameters Default Values

- A formal IN parameter can have a default
- A formal parameter with a default value is called an optional parameter (its corresponding actual parameter is optional in a subprogram invocation)
- If the actual parameter is omitted, then the invocation assigns the default value to the formal parameter
- A formal parameter with no default value is called a required parameter (its corresponding actual parameter is required in a subprogram invocation)
- Omitting an actual parameter does not make the value of the corresponding formal parameter NULL
- To make the value of a formal parameter NULL, specify NULL as either the default value or the actual parameter



Subprogram Parameters
Positional notation

- When invoking a subprogram, a user can specify the actual parameters using either

- Positional notation
  - Specify the actual parameters in the same order as the formal parameters are declared
  - You can omit trailing optional parameters
  - Specifying actual parameters in the wrong order can cause problems that are hard to detect, especially if the actual parameters are literals.
  - Subprogram invocations must change if the formal parameter list changes, unless the list only acquires new trailing optional parameters
  - Reduced code clarity and maintainability
  - Not recommended if the subprogram has a large number of parameters



Subprogram Parameters
Named notation

- Specify the actual parameters in any order, using this syntax:
  formal => actual

- Where
  - **formal** is the name of the formal parameter
  - **actual** is the actual parameter

- You can omit any optional parameters

- There is no wrong order for specifying actual parameters

- Subprogram invocations must change only if the formal parameter list acquires new required parameters

- Recommended when you invoke a subprogram defined or maintained by someone else



Subprogram Parameters
Mixed notation

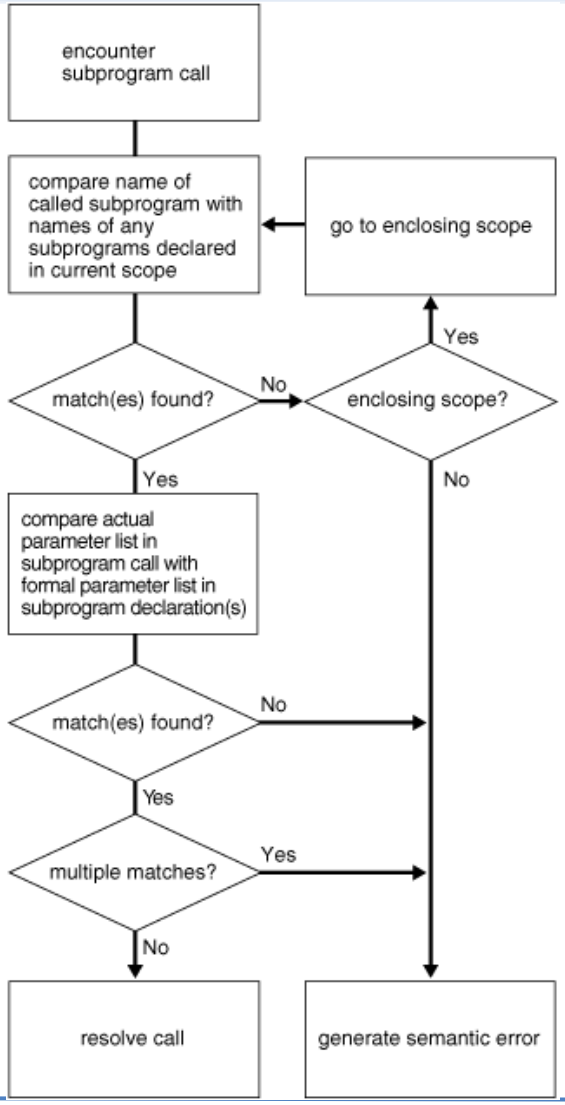
- Start with positional notation, then use named notation for the remaining parameters
- In the positional notation: you can omit trailing optional parameters
- In the named notation: you can omit any optional parameters
- Convenient when you invoke a subprogram that has required parameters followed by optional parameters, and you must specify only a few of the optional parameters.
- In the positional notation, the wrong order can cause problems that are hard to detect, especially if the actual parameters are literals
- Changes to the formal parameter list might require changes in the positional notation



Subprogram Parameters
Default Values and Notations

```
 1 DECLARE
 2   FUNCTION f_pow (
 3     x IN number,
 4     y IN number := 2,
 5     z IN number default 1
 6   ) RETURN NUMBER IS
 7   BEGIN
 8     RETURN (x ** y) ** z;
 9   END f_pow;
10 BEGIN
11   DBMS_OUTPUT.PUT_LINE(f_pow(2));
12   DBMS_OUTPUT.PUT_LINE(f_pow(2, 4));
13   DBMS_OUTPUT.PUT_LINE(f_pow(2, 4, 4));
14   DBMS_OUTPUT.PUT_LINE(f_pow(z => 2, x => 4));
15   DBMS_OUTPUT.PUT_LINE(f_pow(3, 3, z => 4));
16   DBMS_OUTPUT.PUT_LINE(f_pow(5, z => 4));
17   DBMS_OUTPUT.PUT_LINE(f_pow(z => 3, x => 3, y => 4));
18 END;
19 /
```

Subprogram Invocation Resolution





Subprogram Invocation Resolution

- When the PL/SQL compiler encounters a subprogram invocation, it searches for a matching subprogram declaration
  - First in the current scope
  - If necessary, in successive enclosing scopes
- A declaration and invocation match if their subprogram names and parameter lists match
- The parameter lists match if each required formal parameter in the declaration has a corresponding actual parameter in the invocation
- If the compiler finds no matching declaration for an invocation, then it generates a semantic error



Overloaded Subprograms

- PL/SQL permits to overload nested subprograms, package subprograms, and type methods
- The same name can be used for several different subprograms if their formal parameters differ in name, number, order, or data type family
- A data type family is a data type and its subtypes
- If formal parameters differ only in name, then named notation must be used to specify the corresponding actual parameters



Overloaded Subprograms

- Subprograms can be overloaded if their formal parameters differ only in numeric data type

- This technique is useful in writing mathematical application programming interfaces (APIs) (i.e., several versions of a function can use the same name, and each can accept a different numeric type)

- To avoid problems or unexpected results when passing parameters to such overloaded subprograms:
  - Ensure that the expected version of a subprogram is invoked for each set of expected parameters
  - Qualify numeric literals and use conversion functions to make clear what the intended parameter types are



Overloaded Subprograms

- PL/SQL looks for matching numeric parameters in this order:
  - PLS_INTEGER (or BINARY_INTEGER, an identical data type)
  - NUMBER
  - BINARY_FLOAT
  - BINARY_DOUBLE
  - A VARCHAR2 value can match a NUMBER, BINARY_FLOAT, or BINARY_DOUBLE parameter.

- PL/SQL uses the first overloaded subprogram that matches the supplied parameters

- If PL/SQL must convert a parameter to another data type, it first tries to convert it to a higher data type



Overloaded Subprograms DO NOT DO

- Cannot overload these subprograms:
  - Standalone subprograms
  - Subprograms whose formal parameters differ only in mode; e.g.:

    PROCEDURE s (p IN VARCHAR2) IS …
    PROCEDURE s (p OUT VARCHAR2) IS …

  - Subprograms whose formal parameters differ only in subtype; e.g.:

    PROCEDURE s (p INTEGER) IS ...
    PROCEDURE s (p REAL) IS …

    - INTEGER and REAL are subtypes of NUMBER, so they belong to the same data type family.

  - Functions that differ only in return value data type, even if the data types are in different families; e.g.:

    FUNCTION f (p INTEGER) RETURN BOOLEAN IS ...
    FUNCTION f (p INTEGER) RETURN INTEGER IS …



Overloaded Subprograms
Overload Errors

- The PL/SQL compiler catches overload errors as soon as it determines that it cannot tell which subprogram was invoked

- When subprograms have identical headings, the compiler catches the overload error at the compilation time of
  - The subprograms themselves (if they are nested)
  - The package specification that declares them
  - An ambiguous invocation of a subprogram



Overloaded Subprograms
Overload

```
 1 DECLARE
 2   TYPE date_tab_typ IS TABLE OF DATE   INDEX BY PLS_INTEGER;
 3   TYPE num_tab_typ  IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
 4
 5   hiredate_tab  date_tab_typ;
 6   sal_tab       num_tab_typ;
 7
 8   PROCEDURE initialize (tab OUT date_tab_typ, n INTEGER) IS
 9   BEGIN
10     DBMS_OUTPUT.PUT_LINE('Invoked first version');
11     FOR i IN 1..n LOOP
12       tab(i) := SYSDATE;
13     END LOOP;
14   END initialize;
15
16   PROCEDURE initialize (tab OUT num_tab_typ, n INTEGER) IS
17   BEGIN
18     DBMS_OUTPUT.PUT_LINE('Invoked second version');
19     FOR i IN 1..n LOOP
20       tab(i) := 0.0;
21     END LOOP;
22   END initialize;
23
24 BEGIN
25   initialize(hiredate_tab, 50);
26   initialize(sal_tab, 100);
27 END;
28 /
```



Recursive Subprograms

- A recursive subprogram invokes itself
- Recursion is a powerful technique for simplifying an algorithm
- A recursive subprogram must have at least two execution paths
  1. One leading to the recursive invocation
  2. One leading to a terminating condition
  - Without the latter, recursion continues until PL/SQL runs out of memory and raises the predefined exception STORAGE_ERROR
- Each recursive invocation of a subprogram creates an instance of
  - Each item that the subprogram declares
  - Each SQL statement that the subprogram executes
- A recursive invocation inside a cursor FOR LOOP statement, or between an OPEN or OPEN FOR statement and a CLOSE statement
  - Opens another cursor at each invocation
  - Might cause the number of open cursors to exceed the limit set by the database initialization parameter OPEN_CURSORS



Recursive Subprograms

```
 1 CREATE OR REPLACE FUNCTION factorial(n POSITIVE)
 2 RETURN POSITIVE AUTHID DEFINER
 3 IS
 4 BEGIN
 5   IF n = 1 THEN
 6     -- terminating condition
 7     RETURN n;
 8   ELSE
 9     -- recursive invocation
10     RETURN n * factorial(n-1);
11   END IF;
12 END;
13 /
14
15 BEGIN
16   FOR i IN 1..5 LOOP
17     DBMS_OUTPUT.PUT_LINE(i || '! = ' || factorial(i));
18   END LOOP;
19 END;
20 /
```



Subprograms Side Effects

- A subprogram has side effects if it changes anything except the values of its own local variables.

- A subprogram that changes any of the following has side effects:
  - Its own OUT or IN OUT parameter
  - A global variable
  - A public variable in a package
  - A database table
  - The database
  - The external state

- Side effects can
  - Prevent the parallelization of a query
  - Yield order-dependent (and therefore, indeterminate) results
  - Require that package state be maintained across user sessions

- Minimizing side effects is especially important when defining a result-cached function or a stored function for SQL statements to invoke.



PL/SQL Function Result Cache

- When a PL/SQL function has the RESULT_CACHE option, its results are cached in the shared global area (SGA) so sessions connected to the same instance can reuse these results when available

- Oracle Database
  - Automatically detects all data sources (tables and views) that are queried while a result-cached function is running
  - Changes to any of these data sources are committed, the cached result becomes invalid across all instances

- The best candidates for result-caching are functions that are invoked frequently but depend on information that changes infrequently or never

- To make a function result-cached, include the RESULT_CACHE clause in the function declaration and definition



PL/SQL Function Result Cache

```
 1 CREATE OR REPLACE FUNCTION f
 2 RETURN VARCHAR2
 3 RESULT_CACHE
 4 IS
 5   name VARCHAR2(50);
 6 BEGIN
 7   SELECT first_name || ' ' || last_name
 8   INTO name
 9   FROM employees
10   WHERE manager_id IS NULL;
11   RETURN name;
12 END;
13 /
```



PL/SQL Functions Invoked by SQL

- To be invokable from SQL statements, a stored function (and any subprograms that it invokes) must obey the following purity rules, which are meant to control side effects:
  - The subprogram **cannot** modify any database tables when invoked from a SELECT statement or a parallelized INSERT, UPDATE, DELETE, or MERGE statement
  - The subprogram **cannot** query or modify any database tables modified by that statement when invoked from an INSERT, UPDATE, DELETE, or MERGE statement
  - The **subprogram cannot execute** any of the following SQL statements (unless PRAGMA AUTONOMOUS_TRANSACTION was specified) when invoked from a SELECT, INSERT, UPDATE, DELETE, or MERGE statement:
    - Transaction control statements (such as COMMIT)
    - Session control statements (such as SET ROLE)
    - System control statements (such as ALTER SYSTEM)
    - Database definition language (DDL) statements (such as CREATE), which are committed automatically

PL/SQL Functions Invoked by SQL

- If any SQL statement in the execution part of the function violates a rule, then a runtime error occurs when that statement is parsed
- The fewer side effects a function has, the better it can be optimized in a SELECT statement, especially if the function is declared with the option DETERMINISTIC or PARALLEL_ENABLE

```
1 SELECT f CEO_NAME FROM dual;
```



External Subprograms

- If a **C procedure** or **Java method** is **stored** in the database, you can **publish** it **as an external** subprogram
- To **publish** an external subprogram, **define** a stored PL/SQL **subprogram** with a **call specification**
- **Invoke** the published external subprogram by its PL/SQL name



External Subprograms

```
 1 CREATE OR REPLACE JAVA SOURCE NAMED "Hello" AS
 2   /* Pure Java code */
 3   public class Hello {
 4     static public String Msg(String tail) {
 5       return "Hello " + tail;
 6     }
 7   }
 8
 9 CREATE OR REPLACE FUNCTION hello (str varchar2)
10 RETURN varchar
11 AS
12   LANGUAGE JAVA NAME 'Hello.Msg(java.lang.String) return java.lang.String';
13 /
14
15 SELECT hello(first_name|| ' '||last_name) FROM employees;
```



External Subprograms

```
1 CREATE OR REPLACE JAVA SOURCE NAMED "Adjuster" AS
2   import java.sql.*;
3   import oracle.jdbc.driver.*;
4   public class Adjuster {
5     public static void raiseSalary (int empNo, float percent)
6     throws SQLException {
7       Connection conn = new OracleDriver().defaultConnection();
8       String sql = "UPDATE employees SET salary = salary * ? WHERE employee_id = ?";
9       try {
10        PreparedStatement pstmt = conn.prepareStatement(sql);
11        pstmt.setFloat(1, (1 + percent / 100));
12        pstmt.setInt(2, empNo);
13        pstmt.executeUpdate();
14        pstmt.close();
15      } catch (SQLException e)
16           {System.err.println(e.getMessage());}
17      }
18  }
19 /
```



External Subprograms

```
 1 CREATE OR REPLACE PROCEDURE raise_salary (
 2   empid NUMBER,
 3   pct   NUMBER
 4 )
 5 AS -- call specification
 6   LANGUAGE JAVA NAME 'Adjuster.raiseSalary (int, float)';
 7 /
 8
 9 SELECT salary FROM employees WHERE employee_id=120;
10
11 BEGIN
12   -- invoke Adjuster.raiseSalary by PL/SQL name
13   raise_salary(120, 10);
14 END;
15 /
16
17 SELECT salary FROM employees WHERE employee_id=120;
```



Bibliography

- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]